WILLIS PORT

Codex Implementation Brief

Project Direction Reset - Structured Shipment Classification, Database-Authoritative Pricing, Invoicing and Auditability

Prepared from the current Willis Port implementation and the shipment/pricing audit completed during development.

1. Executive Direction

Willis Port should move away from loosely-entered pricing inputs and toward a structured, database-authoritative workflow. Staff should classify a shipment and record operational measurements. The application should resolve the approved pricing rule from the database, calculate charges on the server, store a historical pricing snapshot, and use that approved snapshot for invoicing.

Primary principle: staff chooses the business classification and records factual measurements; the database decides the pricing basis and approved rate.

2. Current Project Assessment

Area	Assessment
Architecture	Next.js App Router + TypeScript + Prisma + PostgreSQL. Local PostgreSQL is the development database; Neon is the review/staging database.
Shipment	Stores current operational facts including shippingMode, serviceType, goodsCategory, weightKg, chargeableWeightKg, declaredCbm, actualCbm and chargeableCbm.
ShipmentPricing	Stores a pricing-time snapshot including pricingBasis, measurement snapshots, billableQuantity, unitRateUsd, manualChargeUsd, extra charges, exchange rate and totals.
ShippingRate	Source of truth for structured pricing basis and approved rate.
Rate lookup key	shippingMode + serviceType + goodsCategory.
Verified local rule	AIR + STANDARD + NORMAL resolves to KG pricing at $17/kg.
Current create-form gap	New shipment creation exposes shippingMode but does not yet expose serviceType or goodsCategory as explicit structured dropdowns.
Pricing progress	The frontend can display approved rates. The server now derives pricing basis from ShippingRate and KG pricing uses Shipment.chargeableWeightKg.
Historical test records	Older local pricing rows may still show CBM from before the KG corrections. Use a completely new shipment for the clean acceptance test.

3. Non-Negotiable Business Rules

- ShippingRate is authoritative for structured pricing. Never accept a browser-supplied CBM/KG unit rate as authoritative.
- The structured rate lookup key is shippingMode + serviceType + goodsCategory.
- Only active ShippingRate records may be used.
- For CBM pricing, the server uses Shipment.chargeableCbm.
- For KG pricing, the server uses Shipment.chargeableWeightKg.
- Manual/Special pricing remains an explicit exception with an approved manual amount.
- ShipmentPricing is historical. Store the exact shipment measurements, rate and billable quantity used at pricing time.

- Once pricing is APPROVED, normal edit/save actions must not silently replace it.
- Extra charges remain explicit before approval: handling, documentation, special handling, delivery and other.
- Freight Charge remains removed from the active workflow. Keep any legacy column only for compatibility and store zero in new records.
- The browser may preview totals, but the server must recalculate the authoritative amount.

4. Shipment Form Redesign

Separate pricing classifications from descriptive shipment information.

Field	Control	Stored On	Purpose
Tracking Number	Text	Shipment	Operational identifier
Shipping Mode	Dropdown: AIR / SEA / UNKNOWN	Shipment.shippingMode	Pricing classification
Service Type	Dropdown: STANDARD / EXPRESS	Shipment.serviceType	Pricing classification
Goods Category	Dropdown: NORMAL / SPECIAL	Shipment.goodsCategory	Pricing classification
Goods Type	Free text	Shipment.goodsType	Descriptive only
Date Received	Date	Shipment.dateReceived	Operational fact
Weight (kg)	Number	Shipment.weightKg	Actual physical weight
Chargeable Weight (kg)	Number	Shipment.chargeableWeightKg	KG billing quantity
Declared CBM	Number	Shipment.declaredCbm	Declared/source volume
Actual CBM	Number	Shipment.actualCbm	Measured volume
Chargeable CBM	Number	Shipment.chargeableCbm	CBM billing quantity
Description	Text area	Shipment.description	Human-readable description

Important: Goods Type is not Goods Category. Never infer NORMAL/SPECIAL from arbitrary Goods Type text. Goods Category must be selected from a controlled dropdown and persisted.

5. Dropdown Behavior

Stage 1 - implement now:

- Shipping Mode: AIR, SEA, UNKNOWN.
- Service Type: STANDARD, EXPRESS.
- Goods Category: NORMAL, SPECIAL.
- On edit, preselect values already stored on the Shipment row.
- On create, require explicit Service Type and Goods Category instead of relying on hidden database defaults.
- Persist submitted values through the shipment POST/PATCH APIs after strict enum validation.

Stage 2 - refinement after the clean test:

- Use active ShippingRate rows to constrain valid combinations.
- If no active SEA + EXPRESS rule exists, disable or omit EXPRESS after SEA is selected.
- Show a clear UI message when no active rate exists for the selected combination.

6. Pricing Resolution Contract

1. Load Shipment by shipmentId.

2. Read Shipment.shippingMode, Shipment.serviceType and Shipment.goodsCategory.

3. Find the unique ShippingRate using the composite key.

4. Reject if the rate does not exist or is inactive.

5. For normal structured pricing, derive pricingBasis from ShippingRate.pricingBasis.

6. If KG, require Shipment.chargeableWeightKg > 0 and use ShippingRate.rateUsd.

7. If CBM, require Shipment.chargeableCbm > 0 and use ShippingRate.rateUsd.

8. Calculate base charge server-side.

9. Add validated extra charges.

10. Apply the exchange rate to produce the GHS total.

11. Store measurements, rate, billable quantity and totals in ShipmentPricing.

7. Direct Instructions to Codex

Inspect the existing implementation first. Preserve working logic and make the smallest coherent changes. Do not replace the architecture or rename established models without necessity.

A. Shared shipment fields

- Update src/app/customers/[id]/shipments/new/ShipmentFields.tsx.
- Add defaults.serviceType and defaults.goodsCategory.
- Add Service Type and Goods Category dropdowns.
- Keep goodsType as a separate free-text descriptive field.
- Keep chargeableWeightKg in the measurement section.

B. New shipment form

- Inspect src/app/customers/[id]/shipments/new/NewShipmentForm.tsx.
- Send serviceType and goodsCategory in the POST JSON body.
- Do not hardcode rate or pricingBasis in this form.

C. New shipment API

- Update src/app/api/customers/[id]/shipments/route.ts.
- Add serviceType and goodsCategory to ShipmentBody.
- Add strict parsers for STANDARD/EXPRESS and NORMAL/SPECIAL.
- Reject invalid values with HTTP 400.
- Persist both values in prisma.shipment.create().

D. Edit shipment flow

- Update edit page/form/API so serviceType and goodsCategory are displayed, prepopulated, editable and saved.
- Preserve weightKg and chargeableWeightKg separately.
- Preserve actualCbm and chargeableCbm separately.

E. Pricing page

- Display Shipping Mode, Service Type and Goods Category clearly.
- Display operational measurements read-only.
- Display the matching active ShippingRate.
- If no active rule exists, prevent normal structured pricing and show a clear error.
- Do not make chargeableWeightKg or chargeableCbm editable on the pricing page.

F. Pricing API

- Preserve server-authoritative rate lookup.
- Remove redundant rate-existence checks after the active-rate check.
- For KG use shipment.chargeableWeightKg.
- For CBM use shipment.chargeableCbm.
- Preserve ShipmentPricing snapshots.
- Continue superseding old DRAFT pricing when a new DRAFT is created.
- Continue blocking new pricing when APPROVED pricing exists.
- Do not reintroduce freightChargeUsd into active totals.

G. Invoice integration

- Use approved ShipmentPricing snapshots for monetary values.
- Do not recalculate issued invoice amounts from mutable current shipment measurements.
- Preserve extra-charge breakdown, exchange rate, USD totals and GHS total.
- Preserve invoice status and sent/payment tracking.

8. Database and Migration Safety

- Do not deploy the earlier destructive version of the chargeable-weight migration.
- The intended migration for Shipment.chargeableWeightKg must only ADD the column and must not DROP Shipment.weightKg.
- The local database currently has both weightKg and chargeableWeightKg after repair.
- Because the migration was edited after being applied locally, verify Prisma migration status/checksum behavior before creating another migration or deploying to Neon.
- Do not run a destructive prisma migrate reset without protecting local data that should be retained.
- Before Neon deployment, inspect the migration SQL manually and run Prisma status against .env.review.
- Use prisma migrate deploy for Neon/review. Do not use migrate dev against Neon.

9. Clean Acceptance Test

After the dropdown/API changes, create a completely new shipment. Do not reuse the old shipment that has historical CBM pricing rows.

Input	Value
Shipping Mode	AIR
Service Type	STANDARD
Goods Category	NORMAL
Goods Type	Electronics
Weight	20 kg
Chargeable Weight	23 kg
Declared CBM	1
Actual CBM	1
Chargeable CBM	1

Expected pricing resolution:

- ShippingRate match: AIR + STANDARD + NORMAL.
- Pricing Basis: KG.
- Approved Rate: $17/kg.
- Billable Quantity: 23.
- Base Charge: 23 x 17 = $391.00.
- With no extra charges, customerChargeUsd = $391.00.
- GHS total = $391.00 multiplied by the entered exchange rate.

Expected ShipmentPricing row after Save Draft:

- pricingBasis = KG.
- weightKg = 20.
- chargeableWeightKg = 23.
- billableQuantity = 23.
- unitRateUsd = 17.
- chargeableCbm may be stored as a snapshot, but must not become the KG billable quantity.

10. Quality Gates

- Run npx prisma format.
- Run npx prisma validate.
- Run npx prisma generate when schema/client changes require it.
- Run npx tsc --noEmit and leave zero TypeScript errors.
- Do not use unsafe any casts to bypass new enum validation.
- Do not trust client-supplied structured pricing rates or billable measurements.
- Verify create, edit, price, Save Draft and Approve flows in the browser.
- Verify the final ShipmentPricing row directly in PostgreSQL.
- Do not deploy to Neon until the clean local acceptance test passes.

11. Explicit Do-Not-Do List

- Do not infer Goods Category from free-text Goods Type.
- Do not allow staff to type/edit the approved KG/CBM rate in the normal workflow.
- Do not use old approved pricing rows to validate the new clean workflow.
- Do not recalculate an issued invoice from changed shipment measurements after pricing approval.
- Do not restore Freight Charge to active pricing totals at this stage.
- Do not hardcode business rates in React components.
- Do not deploy a migration that drops weightKg.
- Do not redesign unrelated areas while implementing this direction change.

12. Definition of Done

This phase is complete when a new shipment can be created with explicit Shipping Mode, Service Type and Goods Category; the edit form prepopulates and saves those values; the pricing page resolves the active ShippingRate; KG/CBM billable quantities come from Shipment; the server saves a correct ShipmentPricing snapshot; pricing can be approved; Prisma and TypeScript checks pass; and the clean local acceptance test produces the expected database values.

Implementation priority: finish structured shipment classification and the clean pricing test first. Only then proceed to Neon review deployment and subsequent invoice refinements.